EXPERIMENT

Demonstrate the working of the decision tree based ID3 algorithm. Use an appropriate data set for building the decision tree and apply this knowledge to classify a new sample.

Name : S.Nagulan

Reg No : 1924260216

(This experiment is divided into 5 problems)

Problem 1: Play Tennis Data Set

Aim

To demonstrate the working of the decision tree based ID3 algorithm by building a decision tree from the play tennis data set (to decide whether tennis can be played under given weather conditions) and to classify a new sample using the constructed tree.

Procedure

1. Store the training examples (4 attributes: Outlook, Temperature, Humidity, Wind; target attribute: PlayTennis) as a list in Python.
2. Compute the entropy of the target attribute for the current set of examples: Entropy(S) = - sum of p(i) log2 p(i).
3. Compute the Information Gain of every remaining attribute: Gain(S, A) = Entropy(S) - sum of (|Sv|/|S|) x Entropy(Sv).
4. Select the attribute with the highest information gain as the decision node and create one branch for each of its values.
5. Repeat steps 2 to 4 recursively on every branch until all examples in a branch have the same class (leaf node) or no attributes remain (majority class).
6. Print the decision tree and traverse it from the root, following the branches that match the attribute values of the new sample, to predict its class.

Coding (Python)

```
# ID3 Decision Tree Algorithm - Play Tennis Data Set
import math
from collections import Counter

names = ['Outlook', 'Temperature', 'Humidity', 'Wind']
target = "PlayTennis"
data = [
    ['Sunny', 'Hot', 'High', 'Weak', 'No'],
    ['Sunny', 'Hot', 'High', 'Strong', 'No'],
    ['Overcast', 'Hot', 'High', 'Weak', 'Yes'],
    ['Rainy', 'Mild', 'High', 'Weak', 'Yes'],
    ['Rainy', 'Cool', 'Normal', 'Weak', 'Yes'],
    ['Rainy', 'Cool', 'Normal', 'Strong', 'No'],
    ['Overcast', 'Cool', 'Normal', 'Strong', 'Yes'],
    ['Sunny', 'Mild', 'High', 'Weak', 'No'],
    ['Sunny', 'Cool', 'Normal', 'Weak', 'Yes'],
    ['Rainy', 'Mild', 'Normal', 'Weak', 'Yes'],
    ['Sunny', 'Mild', 'Normal', 'Strong', 'Yes'],
    ['Overcast', 'Mild', 'High', 'Strong', 'Yes'],
    ['Overcast', 'Hot', 'Normal', 'Weak', 'Yes'],
    ['Rainy', 'Mild', 'High', 'Strong', 'No'],
]

def entropy(rows):
    total = len(rows)
    counts = Counter(r[-1] for r in rows)
    return -sum((c / total) * math.log2(c / total) for c in counts.values())
def info_gain(rows, i):
```

```
    total = len(rows)
    remainder = 0
    for v in set(r[i] for r in rows):
        subset = [r for r in rows if r[i] == v]
        remainder += (len(subset) / total) * entropy(subset)
    return entropy(rows) - remainder

def id3(rows, attrs):
    labels = [r[-1] for r in rows]
    if len(set(labels)) == 1:              # pure node -> leaf
        return labels[0]
    if not attrs:                          # no attribute left -> majority
        return Counter(labels).most_common(1)[0][0]
    best = max(attrs, key=lambda i: info_gain(rows, i))
    tree = {names[best]: {}}
    rest = [a for a in attrs if a != best]
    for v in sorted(set(r[best] for r in rows)):
        subset = [r for r in rows if r[best] == v]
        tree[names[best]][v] = id3(subset, rest)
    return tree

def show(tree, level=0):
    for attr, branches in tree.items():
        for v, sub in branches.items():
            print("    " * level + "[" + attr + " = " + v + "]", end="")
            if isinstance(sub, dict):
                print()
                show(sub, level + 1)
            else:
                print(" --> " + target + " : " + sub)

def classify(tree, sample):
    while isinstance(tree, dict):
        attr = next(iter(tree))
        tree = tree[attr][sample[attr]]
    return tree

print("Entropy of the data set =", round(entropy(data), 4))
print("Information Gain of each attribute:")
for i, a in enumerate(names):
    print("   ", a, "=", round(info_gain(data, i), 4))

tree = id3(data, list(range(len(names))))
print("\nDecision Tree:")
show(tree)

new_sample = {"Outlook": "Sunny", "Temperature": "Cool", "Humidity": "Normal", "Wind": "Strong"}
print("\nNew sample :", new_sample)
print("Predicted class :", classify(tree, new_sample))
```

Input

Training data set (14 examples, 4 attributes, target attribute: PlayTennis):

Outlook	Temperature	Humidity	Wind	PlayTennis
Sunny	Hot	High	Weak	No
Sunny	Hot	High	Strong	No
Overcast	Hot	High	Weak	Yes
Rainy	Mild	High	Weak	Yes
Rainy	Cool	Normal	Weak	Yes

Outlook	Temperature	Humidity	Wind	PlayTennis
Rainy	Cool	Normal	Strong	No
Overcast	Cool	Normal	Strong	Yes
Sunny	Mild	High	Weak	No
Sunny	Cool	Normal	Weak	Yes
Rainy	Mild	Normal	Weak	Yes
Sunny	Mild	Normal	Strong	Yes
Overcast	Mild	High	Strong	Yes
Overcast	Hot	Normal	Weak	Yes
Rainy	Mild	High	Strong	No

New sample to be classified:

Outlook	Temperature	Humidity	Wind
Sunny	Cool	Normal	Strong

Output

```
Entropy of the data set = 0.9403
Information Gain of each attribute:
   Outlook = 0.2467
   Temperature = 0.0292
   Humidity = 0.1518
   Wind = 0.0481

Decision Tree:
[Outlook = Overcast] --> PlayTennis : Yes
[Outlook = Rainy]
   [Wind = Strong] --> PlayTennis : No
   [Wind = Weak] --> PlayTennis : Yes
[Outlook = Sunny]
   [Humidity = High] --> PlayTennis : No
   [Humidity = Normal] --> PlayTennis : Yes

New sample : {'Outlook': 'Sunny', 'Temperature': 'Cool', 'Humidity': 'Normal', 'Wind': 'Strong'}
Predicted class : Yes
```

Result

Thus the ID3 decision tree algorithm was demonstrated successfully. The decision tree for the play tennis data set was built with **Outlook** as the root node (highest information gain), and the new sample was classified as **PlayTennis = Yes**.

Problem 2: Customer Buys Computer Data Set

Aim

To demonstrate the working of the decision tree based ID3 algorithm by building a decision tree from the customer buys computer data set (to decide whether a customer will buy a computer) and to classify a new sample using the constructed tree.

Procedure

1. Store the training examples (4 attributes: Age, Income, Student, CreditRating; target attribute: BuysComputer) as a list in Python.
2. Compute the entropy of the target attribute for the current set of examples: Entropy(S) = - sum of p(i) log2 p(i).
3. Compute the Information Gain of every remaining attribute: Gain(S, A) = Entropy(S) - sum of (|Sv|/|S|) x Entropy(Sv).
4. Select the attribute with the highest information gain as the decision node and create one branch for each of its values.
5. Repeat steps 2 to 4 recursively on every branch until all examples in a branch have the same class (leaf node) or no attributes remain (majority class).
6. Print the decision tree and traverse it from the root, following the branches that match the attribute values of the new sample, to predict its class.

Coding (Python)

```
# ID3 Decision Tree Algorithm - Customer Buys Computer Data Set
import math
from collections import Counter

names = ['Age', 'Income', 'Student', 'CreditRating']
target = "BuysComputer"
data = [
    ['Youth', 'High', 'No', 'Fair', 'No'],
    ['Youth', 'High', 'No', 'Excellent', 'No'],
    ['Middle', 'High', 'No', 'Fair', 'Yes'],
    ['Senior', 'Medium', 'No', 'Fair', 'Yes'],
    ['Senior', 'Low', 'Yes', 'Fair', 'Yes'],
    ['Senior', 'Low', 'Yes', 'Excellent', 'No'],
    ['Middle', 'Low', 'Yes', 'Excellent', 'Yes'],
    ['Youth', 'Medium', 'No', 'Fair', 'No'],
    ['Youth', 'Low', 'Yes', 'Fair', 'Yes'],
    ['Senior', 'Medium', 'Yes', 'Fair', 'Yes'],
    ['Youth', 'Medium', 'Yes', 'Excellent', 'Yes'],
    ['Middle', 'Medium', 'No', 'Excellent', 'Yes'],
    ['Middle', 'High', 'Yes', 'Fair', 'Yes'],
    ['Senior', 'Medium', 'No', 'Excellent', 'No'],
]

def entropy(rows):
    total = len(rows)
    counts = Counter(r[-1] for r in rows)
    return -sum((c / total) * math.log2(c / total) for c in counts.values())
def info_gain(rows, i):
```

```
    total = len(rows)
    remainder = 0
    for v in set(r[i] for r in rows):
        subset = [r for r in rows if r[i] == v]
        remainder += (len(subset) / total) * entropy(subset)
    return entropy(rows) - remainder

def id3(rows, attrs):
    labels = [r[-1] for r in rows]
    if len(set(labels)) == 1:               # pure node -> leaf
        return labels[0]
    if not attrs:                           # no attribute left -> majority
        return Counter(labels).most_common(1)[0][0]
    best = max(attrs, key=lambda i: info_gain(rows, i))
    tree = {names[best]: {}}
    rest = [a for a in attrs if a != best]
    for v in sorted(set(r[best] for r in rows)):
        subset = [r for r in rows if r[best] == v]
        tree[names[best]][v] = id3(subset, rest)
    return tree

def show(tree, level=0):
    for attr, branches in tree.items():
        for v, sub in branches.items():
            print("    " * level + "[" + attr + " = " + v + "]", end="")
            if isinstance(sub, dict):
                print()
                show(sub, level + 1)
            else:
                print(" --> " + target + " : " + sub)

def classify(tree, sample):
    while isinstance(tree, dict):
        attr = next(iter(tree))
        tree = tree[attr][sample[attr]]
    return tree

print("Entropy of the data set =", round(entropy(data), 4))
print("Information Gain of each attribute:")
for i, a in enumerate(names):
    print("   ", a, "=", round(info_gain(data, i), 4))

tree = id3(data, list(range(len(names))))
print("\nDecision Tree:")
show(tree)

new_sample = {"Age": "Youth", "Income": "Medium", "Student": "Yes", "CreditRating": "Fair"}
print("\nNew sample :", new_sample)
print("Predicted class :", classify(tree, new_sample))
```

Input

Training data set (14 examples, 4 attributes, target attribute: BuysComputer):

Age	Income	Student	CreditRating	BuysComputer
Youth	High	No	Fair	No
Youth	High	No	Excellent	No
Middle	High	No	Fair	Yes
Senior	Medium	No	Fair	Yes
Senior	Low	Yes	Fair	Yes

Age	Income	Student	CreditRating	BuysComputer
Senior	Low	Yes	Excellent	No
Middle	Low	Yes	Excellent	Yes
Youth	Medium	No	Fair	No
Youth	Low	Yes	Fair	Yes
Senior	Medium	Yes	Fair	Yes
Youth	Medium	Yes	Excellent	Yes
Middle	Medium	No	Excellent	Yes
Middle	High	Yes	Fair	Yes
Senior	Medium	No	Excellent	No

New sample to be classified:

Age	Income	Student	CreditRating
Youth	Medium	Yes	Fair

Output

```
Entropy of the data set = 0.9403
Information Gain of each attribute:
   Age = 0.2467
   Income = 0.0292
   Student = 0.1518
   CreditRating = 0.0481

Decision Tree:
[Age = Middle] --> BuysComputer : Yes
[Age = Senior]
   [CreditRating = Excellent] --> BuysComputer : No
   [CreditRating = Fair] --> BuysComputer : Yes
[Age = Youth]
   [Student = No] --> BuysComputer : No
   [Student = Yes] --> BuysComputer : Yes

New sample : {'Age': 'Youth', 'Income': 'Medium', 'Student': 'Yes', 'CreditRating': 'Fair'}
Predicted class : Yes
```

Result

Thus the ID3 decision tree algorithm was demonstrated successfully. The decision tree for the customer buys computer data set was built with **Age** as the root node (highest information gain), and the new sample was classified as **BuysComputer = Yes**.

Problem 3: Email Spam Data Set

Aim

To demonstrate the working of the decision tree based ID3 algorithm by building a decision tree from the email spam data set (to decide whether an e-mail is spam) and to classify a new sample using the constructed tree.

Procedure

1. Store the training examples (4 attributes: Sender, HasLink, Attachment, Keyword; target attribute: Spam) as a list in Python.
2. Compute the entropy of the target attribute for the current set of examples: Entropy(S) = - sum of p(i) log2 p(i).
3. Compute the Information Gain of every remaining attribute: Gain(S, A) = Entropy(S) - sum of (|Sv|/|S|) x Entropy(Sv).
4. Select the attribute with the highest information gain as the decision node and create one branch for each of its values.
5. Repeat steps 2 to 4 recursively on every branch until all examples in a branch have the same class (leaf node) or no attributes remain (majority class).
6. Print the decision tree and traverse it from the root, following the branches that match the attribute values of the new sample, to predict its class.

Coding (Python)

```
# ID3 Decision Tree Algorithm - Email Spam Data Set
import math
from collections import Counter

names = ['Sender', 'HasLink', 'Attachment', 'Keyword']
target = "Spam"
data = [
    ['Unknown', 'Yes', 'Yes', 'Offer', 'Yes'],
    ['Unknown', 'Yes', 'No', 'Offer', 'Yes'],
    ['Known', 'No', 'No', 'Meeting', 'No'],
    ['Known', 'Yes', 'No', 'Meeting', 'No'],
    ['Unknown', 'No', 'No', 'Offer', 'Yes'],
    ['Known', 'No', 'Yes', 'Offer', 'No'],
    ['Unknown', 'No', 'Yes', 'Meeting', 'No'],
    ['Unknown', 'Yes', 'No', 'Meeting', 'No'],
    ['Known', 'Yes', 'Yes', 'Offer', 'No'],
    ['Unknown', 'Yes', 'Yes', 'Offer', 'Yes'],
]

def entropy(rows):
    total = len(rows)
    counts = Counter(r[-1] for r in rows)
    return -sum((c / total) * math.log2(c / total) for c in counts.values())

def info_gain(rows, i):
    total = len(rows)
    remainder = 0
    for v in set(r[i] for r in rows):
```

```
        subset = [r for r in rows if r[i] == v]
        remainder += (len(subset) / total) * entropy(subset)
    return entropy(rows) - remainder

def id3(rows, attrs):
    labels = [r[-1] for r in rows]
    if len(set(labels)) == 1:               # pure node -> leaf
        return labels[0]
    if not attrs:                           # no attribute left -> majority
        return Counter(labels).most_common(1)[0][0]
    best = max(attrs, key=lambda i: info_gain(rows, i))
    tree = {names[best]: {}}
    rest = [a for a in attrs if a != best]
    for v in sorted(set(r[best] for r in rows)):
        subset = [r for r in rows if r[best] == v]
        tree[names[best]][v] = id3(subset, rest)
    return tree

def show(tree, level=0):
    for attr, branches in tree.items():
        for v, sub in branches.items():
            print("    " * level + "[" + attr + " = " + v + "]", end="")
            if isinstance(sub, dict):
                print()
                show(sub, level + 1)
            else:
                print(" --> " + target + " : " + sub)

def classify(tree, sample):
    while isinstance(tree, dict):
        attr = next(iter(tree))
        tree = tree[attr][sample[attr]]
    return tree

print("Entropy of the data set =", round(entropy(data), 4))
print("Information Gain of each attribute:")
for i, a in enumerate(names):
    print("   ", a, "=", round(info_gain(data, i), 4))

tree = id3(data, list(range(len(names))))
print("\nDecision Tree:")
show(tree)

new_sample = {"Sender": "Unknown", "HasLink": "No", "Attachment": "No", "Keyword": "Offer"}
print("\nNew sample :", new_sample)
print("Predicted class :", classify(tree, new_sample))
```

Input

Training data set (10 examples, 4 attributes, target attribute: Spam):

Sender	HasLink	Attachment	Keyword	Spam
Unknown	Yes	Yes	Offer	Yes
Unknown	Yes	No	Offer	Yes
Known	No	No	Meeting	No
Known	Yes	No	Meeting	No
Unknown	No	No	Offer	Yes
Known	No	Yes	Offer	No

Sender	HasLink	Attachment	Keyword	Spam
Unknown	No	Yes	Meeting	No
Unknown	Yes	No	Meeting	No
Known	Yes	Yes	Offer	No
Unknown	Yes	Yes	Offer	Yes

New sample to be classified:

Sender	HasLink	Attachment	Keyword
Unknown	No	No	Offer

Output

```
Entropy of the data set = 0.971
Information Gain of each attribute:
   Sender = 0.42
   HasLink = 0.0464
   Attachment = 0.0
   Keyword = 0.42

Decision Tree:
[Sender = Known] --> Spam : No
[Sender = Unknown]
   [Keyword = Meeting] --> Spam : No
   [Keyword = Offer] --> Spam : Yes

New sample : {'Sender': 'Unknown', 'HasLink': 'No', 'Attachment': 'No', 'Keyword': 'Offer'}
Predicted class : Yes
```

Result

Thus the ID3 decision tree algorithm was demonstrated successfully. The decision tree for the email spam data set was built with **Sender** as the root node (highest information gain), and the new sample was classified as **Spam = Yes**.

Problem 4: Student Result Data Set

Aim

To demonstrate the working of the decision tree based ID3 algorithm by building a decision tree from the student result data set (to decide whether a student passes the examination) and to classify a new sample using the constructed tree.

Procedure

1. Store the training examples (4 attributes: Attendance, StudyHours, Assignments, PriorMarks; target attribute: Result) as a list in Python.
2. Compute the entropy of the target attribute for the current set of examples: Entropy(S) = - sum of p(i) log2 p(i).
3. Compute the Information Gain of every remaining attribute: Gain(S, A) = Entropy(S) - sum of (|Sv|/|S|) x Entropy(Sv).
4. Select the attribute with the highest information gain as the decision node and create one branch for each of its values.
5. Repeat steps 2 to 4 recursively on every branch until all examples in a branch have the same class (leaf node) or no attributes remain (majority class).
6. Print the decision tree and traverse it from the root, following the branches that match the attribute values of the new sample, to predict its class.

Coding (Python)

```
# ID3 Decision Tree Algorithm - Student Result Data Set
import math
from collections import Counter

names = ['Attendance', 'StudyHours', 'Assignments', 'PriorMarks']
target = "Result"
data = [
    ['High', 'High', 'Done', 'Good', 'Pass'],
    ['High', 'Low', 'Done', 'Good', 'Pass'],
    ['Low', 'High', 'Pending', 'Poor', 'Fail'],
    ['Low', 'Low', 'Pending', 'Poor', 'Fail'],
    ['High', 'High', 'Done', 'Poor', 'Pass'],
    ['Low', 'High', 'Done', 'Good', 'Pass'],
    ['High', 'Low', 'Pending', 'Poor', 'Fail'],
    ['Low', 'Low', 'Done', 'Poor', 'Fail'],
    ['High', 'High', 'Pending', 'Good', 'Pass'],
    ['Low', 'Low', 'Done', 'Good', 'Pass'],
]

def entropy(rows):
    total = len(rows)
    counts = Counter(r[-1] for r in rows)
    return -sum((c / total) * math.log2(c / total) for c in counts.values())

def info_gain(rows, i):
    total = len(rows)
    remainder = 0
    for v in set(r[i] for r in rows):
```

```
        subset = [r for r in rows if r[i] == v]
        remainder += (len(subset) / total) * entropy(subset)
    return entropy(rows) - remainder

def id3(rows, attrs):
    labels = [r[-1] for r in rows]
    if len(set(labels)) == 1:               # pure node -> leaf
        return labels[0]
    if not attrs:                           # no attribute left -> majority
        return Counter(labels).most_common(1)[0][0]
    best = max(attrs, key=lambda i: info_gain(rows, i))
    tree = {names[best]: {}}
    rest = [a for a in attrs if a != best]
    for v in sorted(set(r[best] for r in rows)):
        subset = [r for r in rows if r[best] == v]
        tree[names[best]][v] = id3(subset, rest)
    return tree

def show(tree, level=0):
    for attr, branches in tree.items():
        for v, sub in branches.items():
            print("    " * level + "[" + attr + " = " + v + "]", end="")
            if isinstance(sub, dict):
                print()
                show(sub, level + 1)
            else:
                print(" --> " + target + " : " + sub)

def classify(tree, sample):
    while isinstance(tree, dict):
        attr = next(iter(tree))
        tree = tree[attr][sample[attr]]
    return tree

print("Entropy of the data set =", round(entropy(data), 4))
print("Information Gain of each attribute:")
for i, a in enumerate(names):
    print("   ", a, "=", round(info_gain(data, i), 4))

tree = id3(data, list(range(len(names))))
print("\nDecision Tree:")
show(tree)

new_sample = {"Attendance": "High", "StudyHours": "Low", "Assignments": "Done", "PriorMarks": "Poor"}
print("\nNew sample :", new_sample)
print("Predicted class :", classify(tree, new_sample))
```

Input

Training data set (10 examples, 4 attributes, target attribute: Result):

Attendance	StudyHours	Assignments	PriorMarks	Result
High	High	Done	Good	Pass
High	Low	Done	Good	Pass
Low	High	Pending	Poor	Fail
Low	Low	Pending	Poor	Fail
High	High	Done	Poor	Pass
Low	High	Done	Good	Pass

Attendance	StudyHours	Assignments	PriorMarks	Result
High	Low	Pending	Poor	Fail
Low	Low	Done	Poor	Fail
High	High	Pending	Good	Pass
Low	Low	Done	Good	Pass

New sample to be classified:

Attendance	StudyHours	Assignments	PriorMarks
High	Low	Done	Poor

Output

```
Entropy of the data set = 0.971
Information Gain of each attribute:
   Attendance = 0.1245
   StudyHours = 0.1245
   Assignments = 0.2564
   PriorMarks = 0.61

Decision Tree:
[PriorMarks = Good] --> Result : Pass
[PriorMarks = Poor]
   [Attendance = High]
      [StudyHours = High] --> Result : Pass
      [StudyHours = Low] --> Result : Fail
   [Attendance = Low] --> Result : Fail

New sample : {'Attendance': 'High', 'StudyHours': 'Low', 'Assignments': 'Done', 'PriorMarks': 'Poor'}
Predicted class : Fail
```

Result

Thus the ID3 decision tree algorithm was demonstrated successfully. The decision tree for the student result data set was built with **PriorMarks** as the root node (highest information gain), and the new sample was classified as **Result = Fail**.

Problem 5: Loan Approval Data Set

Aim

To demonstrate the working of the decision tree based ID3 algorithm by building a decision tree from the loan approval data set (to decide whether a bank loan application is approved) and to classify a new sample using the constructed tree.

Procedure

1. Store the training examples (4 attributes: Income, Credit, Employment, Property; target attribute: Approved) as a list in Python.
2. Compute the entropy of the target attribute for the current set of examples: Entropy(S) = - sum of p(i) log2 p(i).
3. Compute the Information Gain of every remaining attribute: Gain(S, A) = Entropy(S) - sum of (|Sv|/|S|) x Entropy(Sv).
4. Select the attribute with the highest information gain as the decision node and create one branch for each of its values.
5. Repeat steps 2 to 4 recursively on every branch until all examples in a branch have the same class (leaf node) or no attributes remain (majority class).
6. Print the decision tree and traverse it from the root, following the branches that match the attribute values of the new sample, to predict its class.

Coding (Python)

```
# ID3 Decision Tree Algorithm - Loan Approval Data Set
import math
from collections import Counter

names = ['Income', 'Credit', 'Employment', 'Property']
target = "Approved"
data = [
    ['High', 'Good', 'Salaried', 'Owned', 'Yes'],
    ['Low', 'Poor', 'Unemployed', 'Rented', 'No'],
    ['High', 'Good', 'Self-Employed', 'Owned', 'Yes'],
    ['Low', 'Good', 'Salaried', 'Owned', 'Yes'],
    ['High', 'Poor', 'Salaried', 'Rented', 'No'],
    ['Low', 'Poor', 'Salaried', 'Owned', 'No'],
    ['High', 'Good', 'Salaried', 'Rented', 'Yes'],
    ['Low', 'Good', 'Unemployed', 'Rented', 'No'],
    ['High', 'Poor', 'Self-Employed', 'Owned', 'No'],
    ['Low', 'Good', 'Self-Employed', 'Owned', 'Yes'],
]

def entropy(rows):
    total = len(rows)
    counts = Counter(r[-1] for r in rows)
    return -sum((c / total) * math.log2(c / total) for c in counts.values())

def info_gain(rows, i):
    total = len(rows)
    remainder = 0
    for v in set(r[i] for r in rows):
```

```
        subset = [r for r in rows if r[i] == v]
        remainder += (len(subset) / total) * entropy(subset)
    return entropy(rows) - remainder

def id3(rows, attrs):
    labels = [r[-1] for r in rows]
    if len(set(labels)) == 1:               # pure node -> leaf
        return labels[0]
    if not attrs:                           # no attribute left -> majority
        return Counter(labels).most_common(1)[0][0]
    best = max(attrs, key=lambda i: info_gain(rows, i))
    tree = {names[best]: {}}
    rest = [a for a in attrs if a != best]
    for v in sorted(set(r[best] for r in rows)):
        subset = [r for r in rows if r[best] == v]
        tree[names[best]][v] = id3(subset, rest)
    return tree

def show(tree, level=0):
    for attr, branches in tree.items():
        for v, sub in branches.items():
            print("    " * level + "[" + attr + " = " + v + "]", end="")
            if isinstance(sub, dict):
                print()
                show(sub, level + 1)
            else:
                print(" --> " + target + " : " + sub)

def classify(tree, sample):
    while isinstance(tree, dict):
        attr = next(iter(tree))
        tree = tree[attr][sample[attr]]
    return tree

print("Entropy of the data set =", round(entropy(data), 4))
print("Information Gain of each attribute:")
for i, a in enumerate(names):
    print("   ", a, "=", round(info_gain(data, i), 4))

tree = id3(data, list(range(len(names))))
print("\nDecision Tree:")
show(tree)

new_sample = {"Income": "Low", "Credit": "Good", "Employment": "Salaried", "Property": "Rented"}
print("\nNew sample :", new_sample)
print("Predicted class :", classify(tree, new_sample))
```

Input

Training data set (10 examples, 4 attributes, target attribute: Approved):

Income	Credit	Employment	Property	Approved
High	Good	Salaried	Owned	Yes
Low	Poor	Unemployed	Rented	No
High	Good	Self-Employed	Owned	Yes
Low	Good	Salaried	Owned	Yes
High	Poor	Salaried	Rented	No
Low	Poor	Salaried	Owned	No

Income	Credit	Employment	Property	Approved
High	Good	Salaried	Rented	Yes
Low	Good	Unemployed	Rented	No
High	Poor	Self-Employed	Owned	No
Low	Good	Self-Employed	Owned	Yes

New sample to be classified:

Income	Credit	Employment	Property
Low	Good	Salaried	Rented

Output

```
Entropy of the data set = 1.0
Information Gain of each attribute:
   Income = 0.029
   Credit = 0.61
   Employment = 0.239
   Property = 0.1245

Decision Tree:
[Credit = Good]
   [Employment = Salaried] --> Approved : Yes
   [Employment = Self-Employed] --> Approved : Yes
   [Employment = Unemployed] --> Approved : No
[Credit = Poor] --> Approved : No

New sample : {'Income': 'Low', 'Credit': 'Good', 'Employment': 'Salaried', 'Property': 'Rented'}
Predicted class : Yes
```

Result

Thus the ID3 decision tree algorithm was demonstrated successfully. The decision tree for the loan approval data set was built with **Credit** as the root node (highest information gain), and the new sample was classified as **Approved = Yes**.